

INFORME - TCC - WORKFLOW

Boas Práticas de Pull Requests

Como a equipe deve abrir, revisar e mergear PRs - com o plano de ação para as próximas três sessões.

POR QUE USAR PRS EM VEZ DE COMMITAR DIRETO NO MASTER

Commitar direto no `master` funciona quando você trabalha sozinho. Em equipe, o problema é que ninguém revisa o código antes de entrar - um bug ou uma quebra de arquitetura vai parar no master sem que ninguém perceba.

A PR cria uma **janela de revisão**: o CI roda, o code review da IA comenta, e pelo menos uma pessoa do time lê o diff antes de mergear. Isso o que transforma um projeto de faculdade em algo que parece profissional.

FLUXO PADRÃO DE UMA PR

01

Crie uma branch com nome descritivo

Use o padrão `tipo/o-que-voce-fez`. Branch com nome genérico é branch que ninguém sabe o que contém.

```
# bom
```

```
# ruim
```

02

Commits pequenos e bem descritos

Cada commit deve fazer uma coisa sã³. O tÃtulo segue o padrÃ£o Conventional Commits (veja a seÃ§Ã£o abaixo).

03

Abra a PR com tÃtulo e descriÃ§Ã£o

O tÃtulo resume o que muda. A descriÃ§Ã£o explica o *porquÃª* – o que motivou essa mudanÃ§a, o que foi decidido, o que ficou de fora e por quÃª.

04

CI e AI review rodam automaticamente

O GitHub Actions vai buildar e testar. O review da IA vai comentar no diff. Leia os comentÃ¡rios antes de pedir review humano.

05

Pelo menos uma pessoa do time revisa

NÃ£o precisa ser longa – olhar o diff e comentar "ok" jÃ¡ conta. O importante Ã© que outra pessoa passou os olhos.

06

Merge com Squash (recomendado)

Squash merge junta todos os commits da branch em um Ãºnico commit no master. O histÃ³rico fica limpo.

CONVENTIONAL COMMITS – O PADRÃO DE MENSAGEM

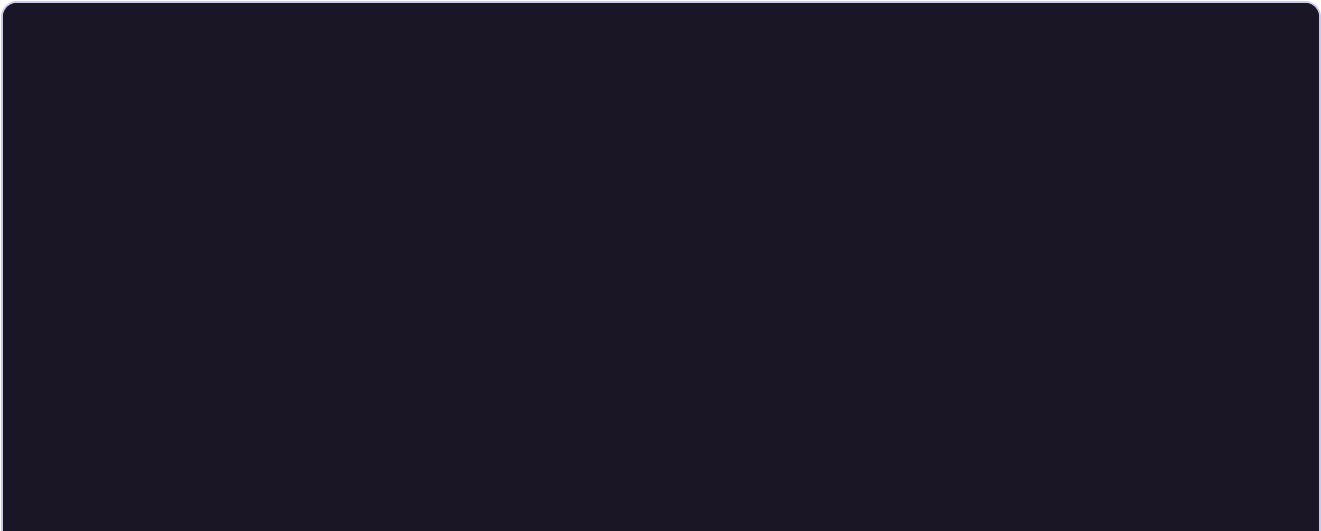
O formato Ã© simples: `tipo(escopo): descriÃ§Ã£o em minÃºsculo`. O escopo Ã© opcional mas recomendado quando identifica o arquivo ou mÃ³dulo alterado.

TIPO	QUANDO USAR	EXEMPLO
feat	Nova funcionalidade ou mÃ©todo	feat(Session): adicionar mÃ©todo Revoke()
fix	CorreÃ§Ã£o de bug	fix(User): trocar DateTimeOffset.Now por .UtcNow
test	Adicionar ou corrigir testes	test(SessionTests): atualizar apÃ³s mudanÃ§a de assinatura
refactor	Reestruturar sem mudar comportamento	refactor(Group): extrair ValidateName para mÃ©todo estÃ¡tico
docs	DocumentaÃ§Ã£o	docs(ARCHITECTURE): detalhar camada de Infrastructure
ci	Pipelines e GitHub Actions	ci: remover MAUI do workflow por ora
chore	Tarefas de manutenÃ§Ã£o (atualizar pacotes, etc.)	chore: atualizar Microsoft.NET.Test.Sdk

Regra de ouro: a mensagem deve completar a frase "Se aplicado, esse commit vaiâ€¦" -> "feat(Session): adicionar mÃ©todo Revoke()" -> "Se aplicado, esse commit vai adicionar o mÃ©todo Revoke() na Session." Funciona. "alteraÃ§Ãµes gabriel" -> nÃ£o funciona.

— COMO ESCREVER UMA BOA DESCRIÃ§Ã£o DE PR

Template recomendado



Exemplo de PR bem descrita

Exemplo de PR que não ajuda ninguém

Quem vai revisar essa PR não sabe o que mudou, não sabe se quebra algo, não sabe como testar. Não há garantia de merge sem revisão de verdade.

COMO REVISAR UMA PR (SEM SER CHATO)

Revisar não é achar culpado é proteger o projeto. O objetivo é que o código que vai para o master seja o melhor que a equipe consegue produzir naquele momento.

O código faz o que o título da PR diz que faz?

Tem algum bug óbvio caso não tratado, null não verificado?

A entidade nova tem testes? O método novo tem pelo menos um teste?

Violou alguma regra da Onion? (Domain referenciando Infrastructure?)

Alguna data usando `.Now` em vez de `.UtcNow` ?

O CI passou? Se não, por quê?

Não corrija estilo ("prefiro aspas simples") é isso bikeshedding

Não peça para refatorar o que não está na scope da PR

Não aprove sem ler é "LGTM" sem leitura é pior do que não revisar

PLANO DE AÇÃO PRÓXIMAS SESSÕES

Com base nas mudanças do repositório e no estado atual do projeto.

SEGUNDA-FEIRA

Correções de Domain

1 hora

QUARTA-FEIRA

Métodos de Domain

1 hora

🔗 Trocar `DateTimeOffset.Now` por `.UtcNow` em todas as entidades `User`, `Session`, `Group`, `AuditLog`, `DeviceDailyMetrics`, `NetworkGrowthSnapshot` - 6 arquivos. Abrir uma PR - para isso.

🔗 Remover construtor manual de `Id` no `AuditLog`
O BD gera o UUID via `gen_random_uuid()`. Passar `Guid` no construtor - desnecessário e perigoso.

🔗 Revisar regex de email
Regex atual ainda não valida `.gov.br` corretamente - precisa de teste com `[Theory]` antes de corrigir.

🔗 Praticar o fluxo de PR
Cada integrante faz a correção em uma branch separada e abre PR - não commitar direto no master.

🔗 Implementar `Session.Revoke()` e `Session.IsExpired()`
Desbloqueiam os TODOs nos `SessionTests` - implementar e descomentar teste na mesma PR.

🔗 Implementar `Device.MarkOnline()` e `Device.MarkOffline()`
Mudam o Status para Online/Offline e atualizam `LastSeenAt`.

🔗 Implementar `EnrollmentToken.Redeem()`
`IsUsed` - true, preenche `RedeemedByDeviceId`. Deve lançar exceção se já foi usado.

🔗 Escrever testes para cada método implementado
O teste entra na mesma PR que o método - não na próxima sprint.

SEXTA-FEIRA

Application Layer - Use Cases

3 horas

🔗 Hora 1 - Alinhamento e design dos use cases

Revisar as interfaces existentes (`IUserRepository`, `IDeviceRepository` etc.) e definir em grupo quais use cases implementar primeiro: `LoginUseCase`, `EnrollDeviceUseCase`, `RecordTelemetryUseCase`. Cada use case vive em `Application/UseCases/`. Sem código ainda - só design.

🔗 Hora 2 - Implementar LoginUseCase

Recebe `username+senha`, usa `IUserRepository.FindByUsernameAsync`, `IPasswordHasher.Verify`, `IJwtTokenService.CreateAccessToken`. Retorna o token ou lança exceção de credencial inválida. Teste no `Application.Tests` mockando as interfaces.

🔗 Hora 3 - Implementar EnrollDeviceUseCase

Valida o `EnrollmentToken` via repositório, cria o `Device` com `AgentKeyHash`, chama `token.Redeem()`, persiste via `IUnitOfWork`. Este é o fluxo RF05-06 - o mais crítico da autenticação do agente.

🔗 Meta: abrir PR dos use cases antes de acabar a sessão
Mesmo que incompleto - PR aberta = feedback antes de mergear. Commitar direto no master não conta.

Regra da sexta: a implementação dos use cases vai exigir mocks (simular o repositório sem banco). Use **NSubstitute** para adicionar o pacote ao `informE.Application.Tests.csproj` antes da sessão para não perder tempo na hora. O comando é `dotnet add tests/informE.Application.Tests package NSubstitute`.